



CSE 2215: Data Structure and Algorithms-I

Array Mapping, Linear Search and
Binary Search



ARRAY MAPPING

Array Mapping

- Mapping 1D arrays
- Mapping 2D arrays

Mapping 1D arrays

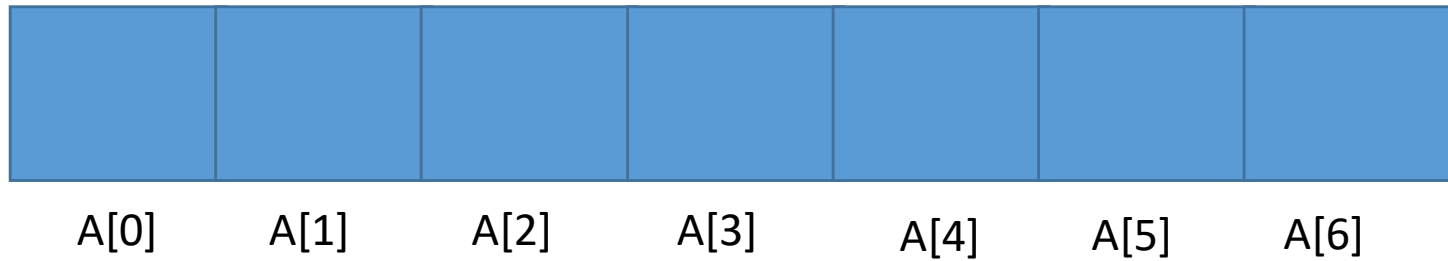
Formula:

$$\text{addr}(a[i]) = b + (i - l_1) \times L$$

Let's do some examples

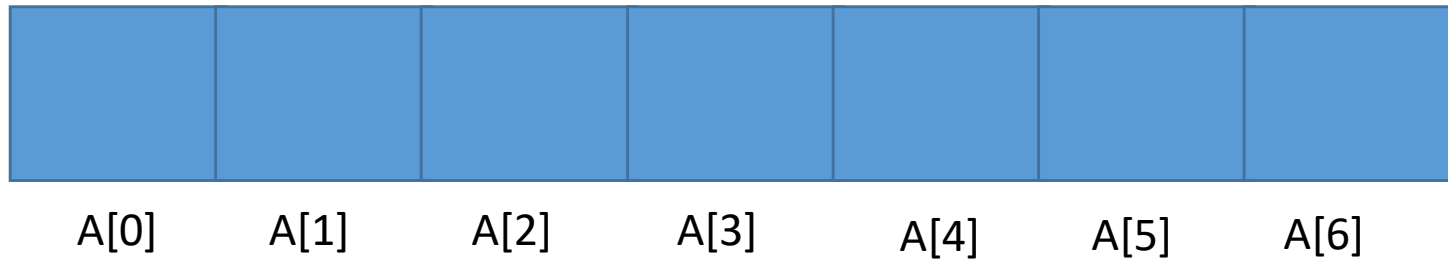
Mapping 1D Array

int A[7]



Mapping 1D Array

int A[7]



10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

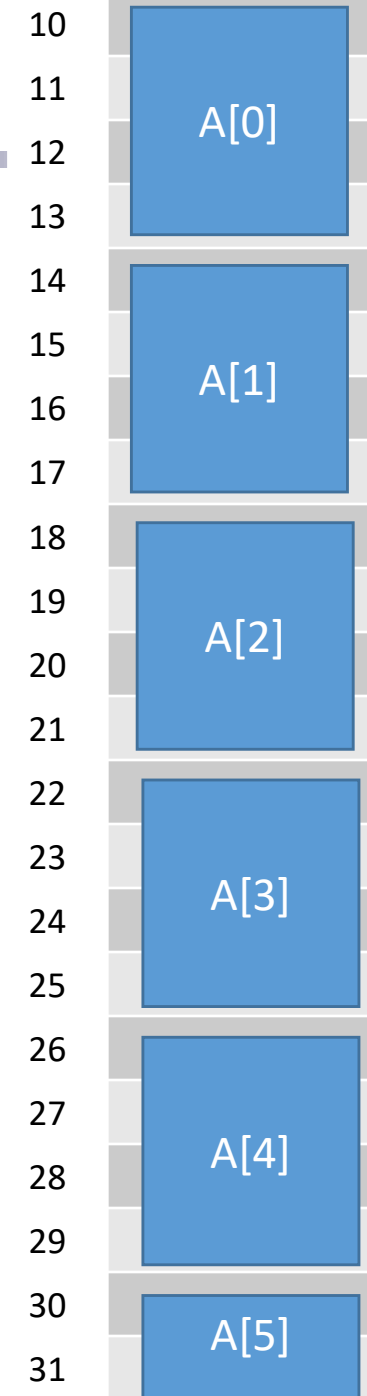
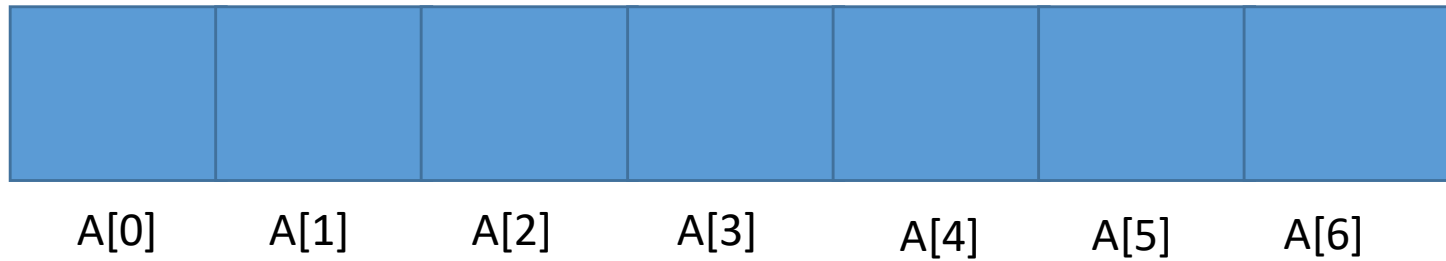
29

30

31

Mapping 1D Array

int A[7]



Mapping 1D Array

Let's do another example:

Mapping 2D Arrays

2 D Arrays:

- Storing in Row Major Order
- Storing in Column Major Order

Mapping 2D Arrays

int A [3] [4]

- How we visualize it.



Mapping 2D Arrays

Row Major Order:



10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

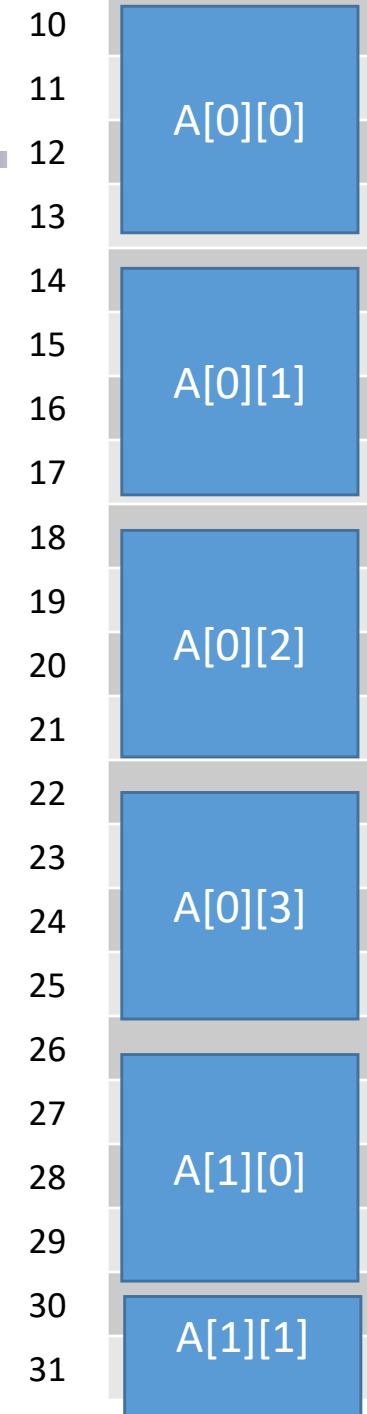
29

30

31

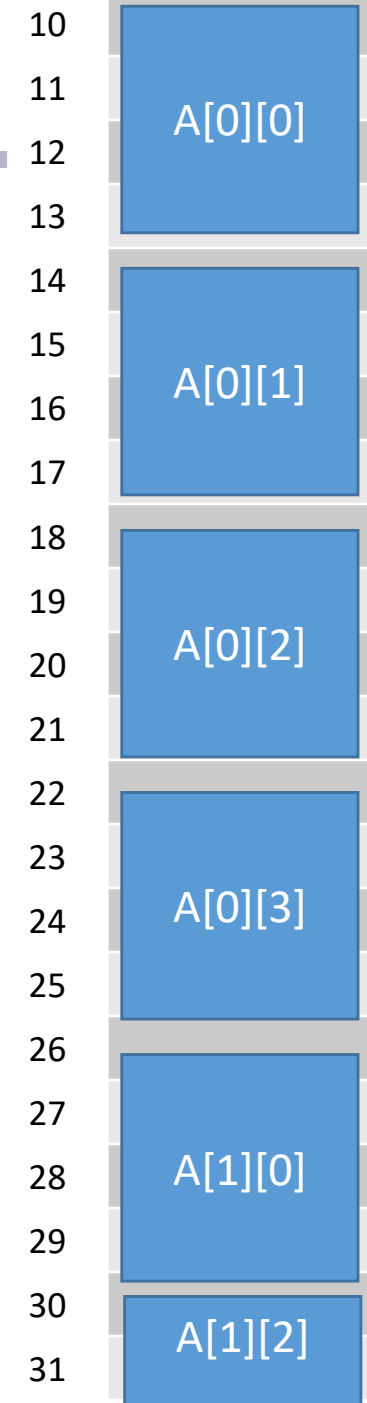
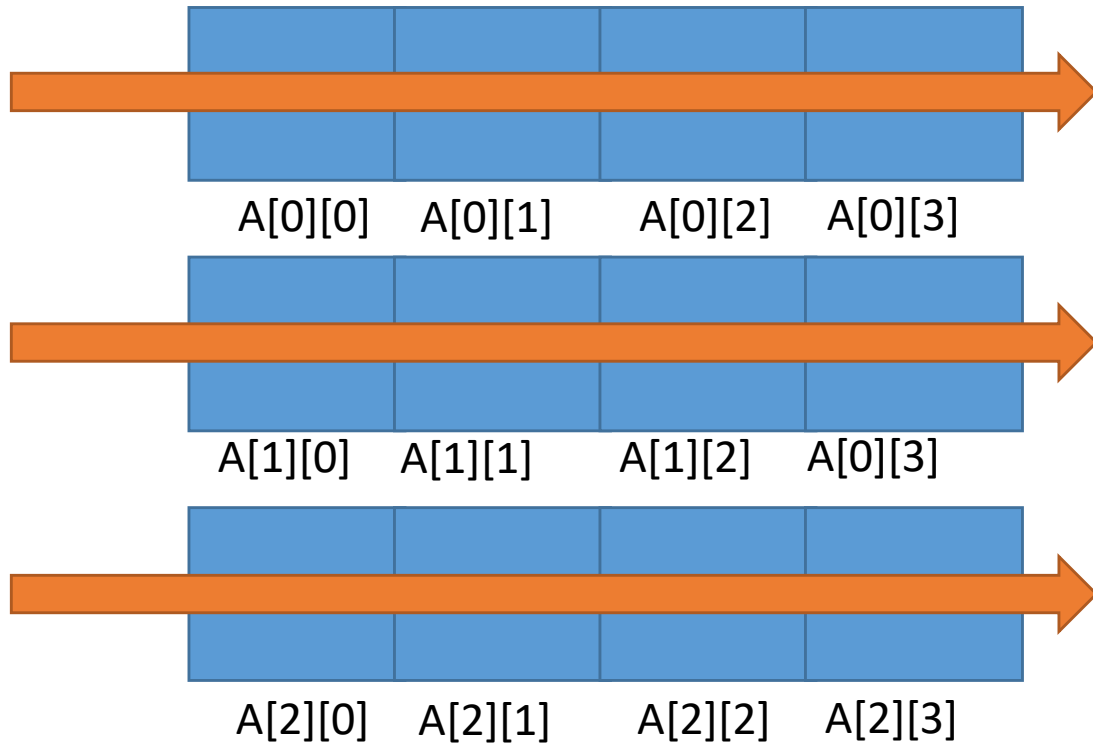
Mapping 2D Arrays

Row Major Order:



Mapping 2D Arrays

Row Major Order:



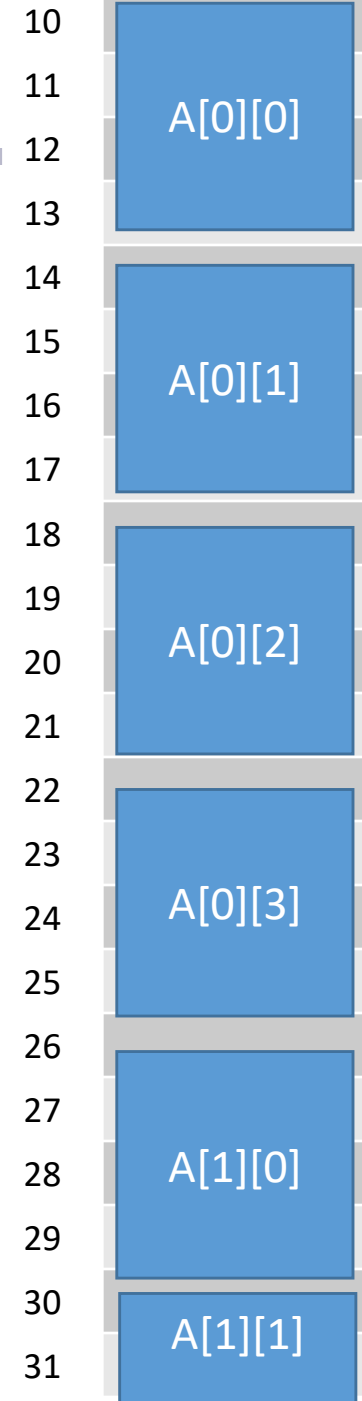
Mapping 2D Arrays

Row Major Order:



$\text{addr}(a[i, j]) =$

$$b + (i - l_1) \times (u_2 - l_2 + 1) \times L + (j - l_2) \times L$$



Mapping 2D Arrays

Col Major Order:



10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

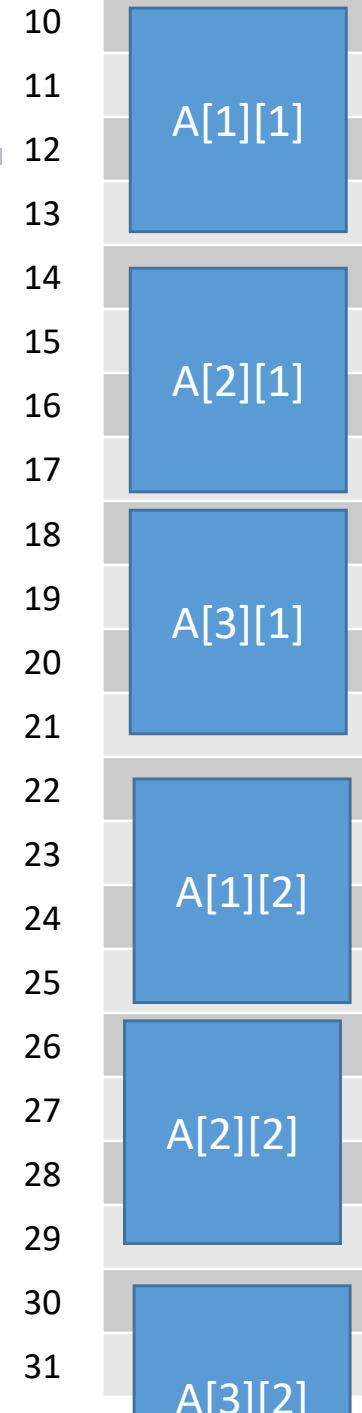
29

30

31

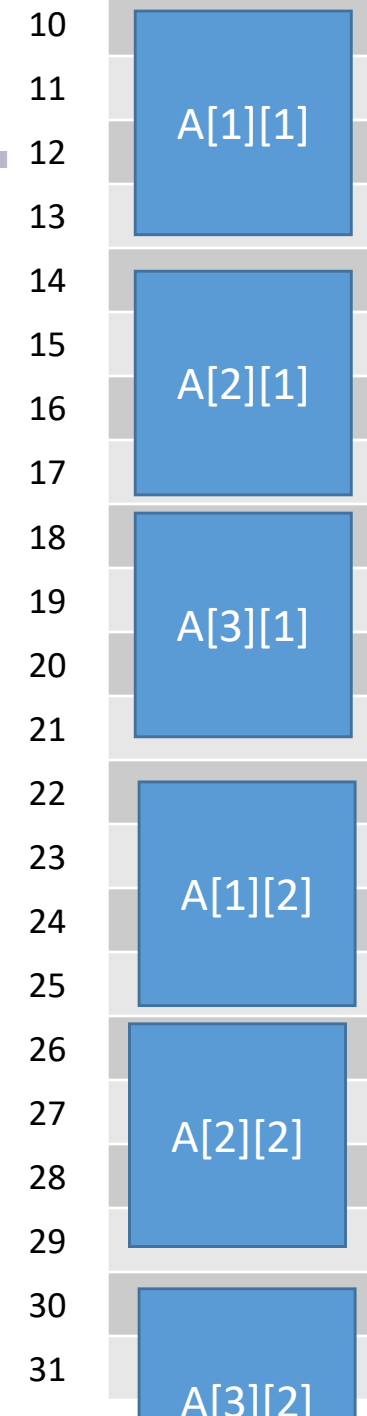
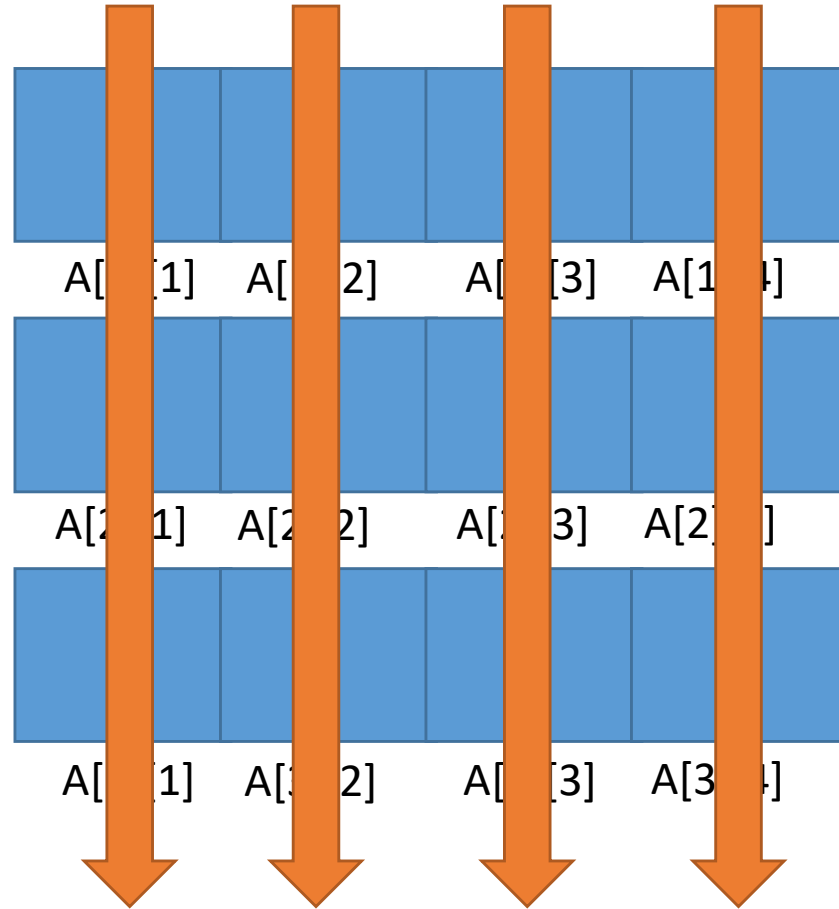
Mapping 2D Arrays

Col Major Order:



Mapping 2D Arrays

Col Major Order:



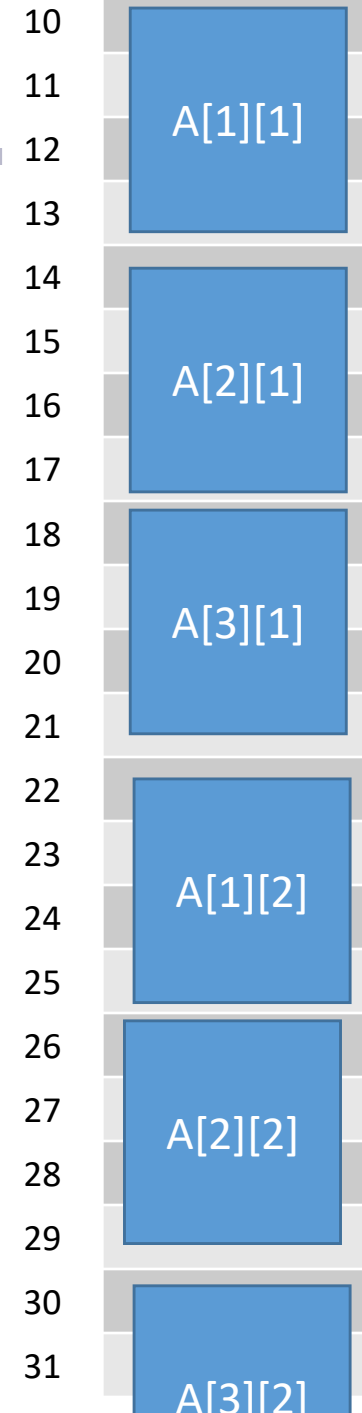
Mapping 2D Arrays

Col Major Order:



$\text{addr}(a[i, j]) =$

$$b + (j - l_2) \times (u_1 - l_1 + 1) \times L + (i - l_1) \times L$$



Mapping 3D Array

$$\text{addr}(a[i, j, k]) = b + (i - l_1) \times (u_2 - l_2 + 1) \times (u_3 - l_3 + 1) \times L + \\ (j - l_2) \times (u_3 - l_3 + 1) \times L + (k - l_3) \times L$$

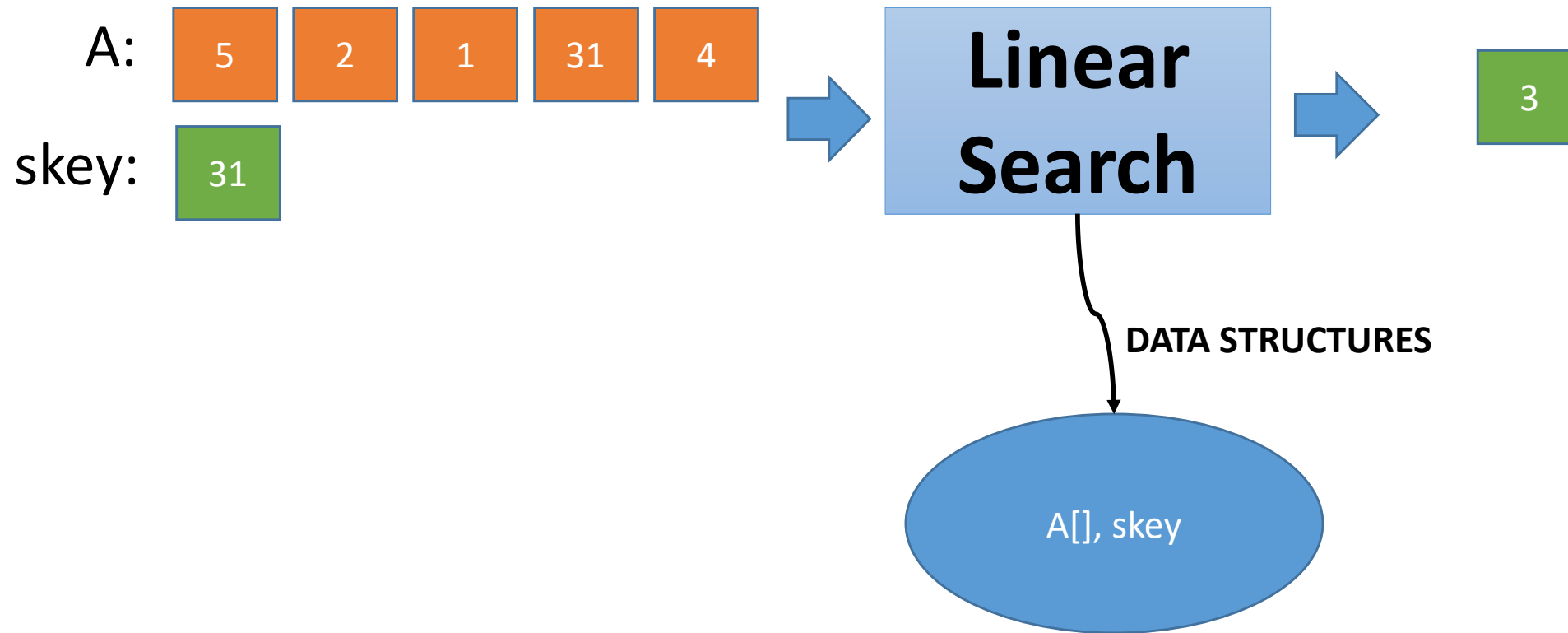
What about 4D, 5D, 6D,...ND?



SEARCHING

Linear Search and Binary Search

Linear Search



Linear Search

```
int LinearSearch (int b[ ], int skey) {  
    int i;  
    for (i=0; i < SIZE; i++)  
        if(b[i] == skey)  
            return i;  
    return -1;  
}
```

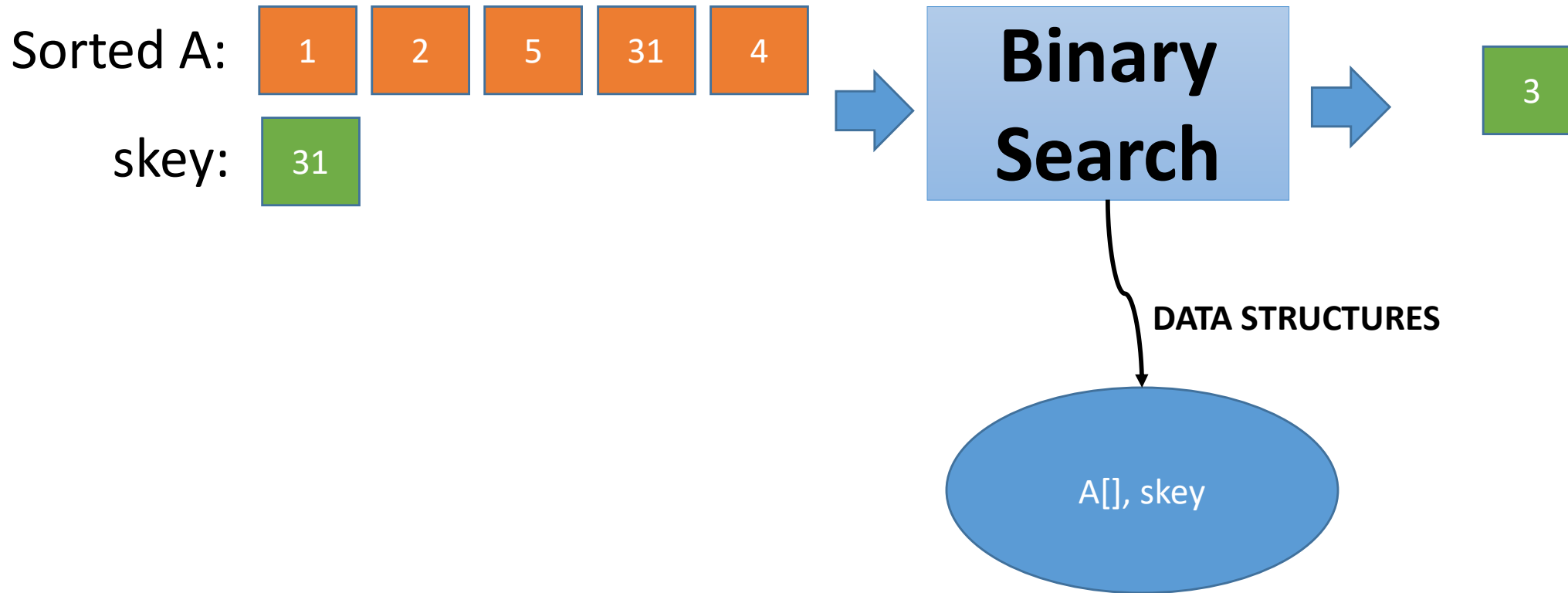
Time Complexity Analysis

We can show that:

- Worst Case : $O(n)$
- Average Case : $O(n)$
- Best Case : $O(1)$

Why are we not using Big Omega or Big Theta?

Binary Search



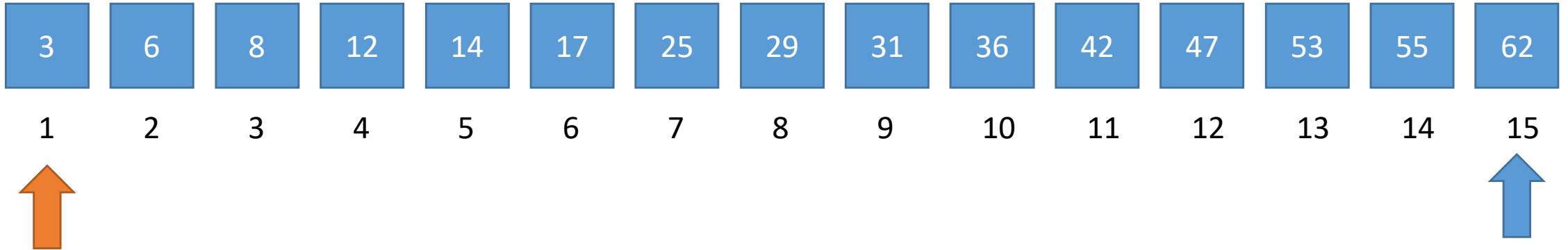
Binary Search

Key=53

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

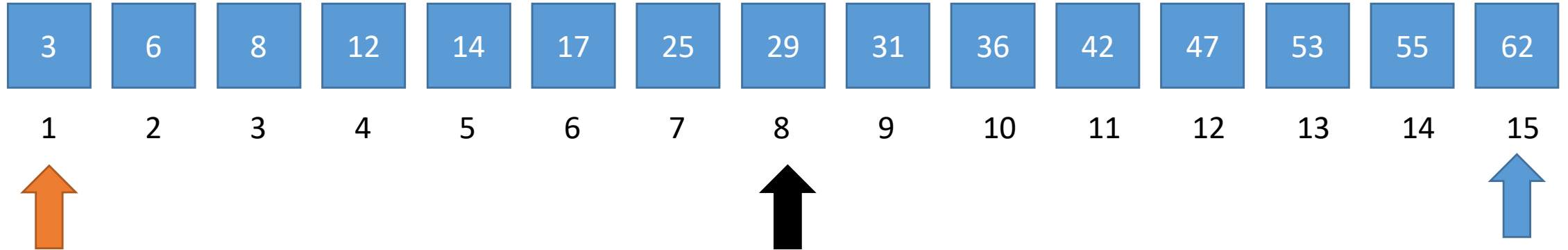
Binary Search

Key=53



Binary Search

Key=53



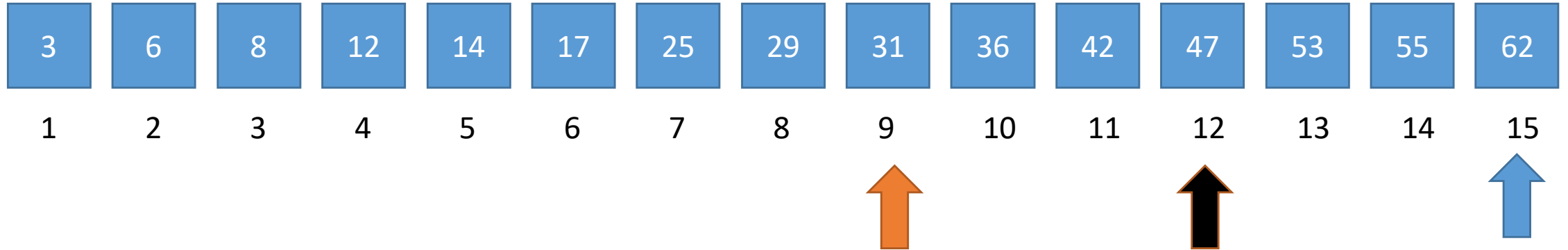
Binary Search

Key=53

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
								↑						↑

Binary Search

Key=53



Binary Search

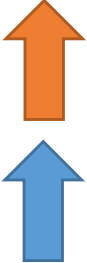
Key=53

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
												↑	↑	↑

Binary Search

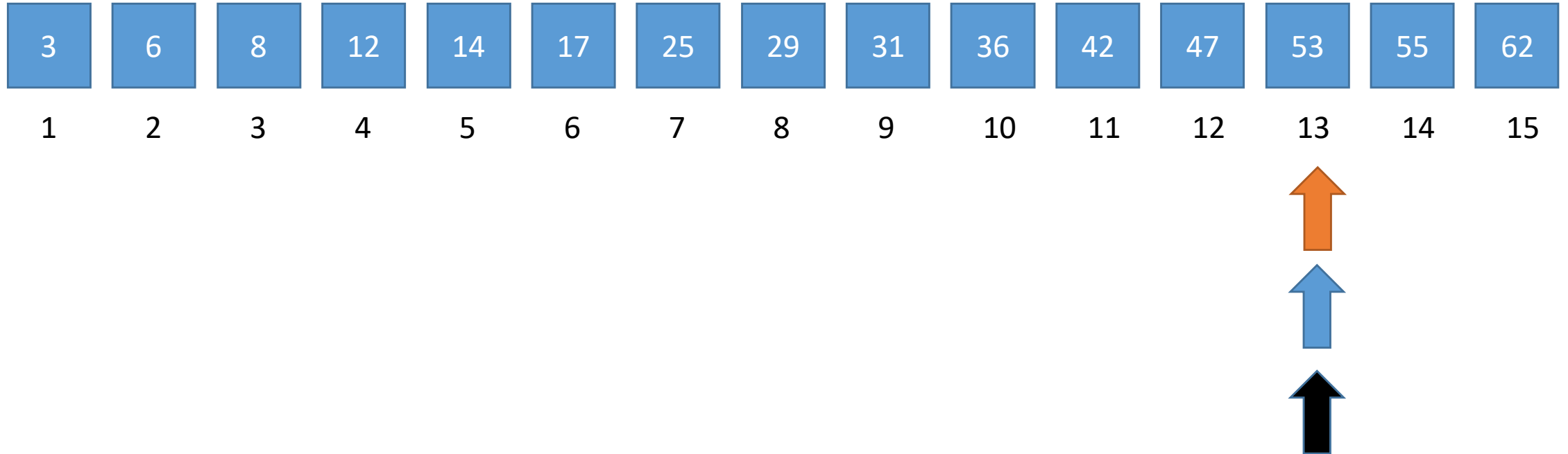
Key=53

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Binary Search

Key=53



Binary Search

Key=53

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

RETURN 13



Binary Search(Another Example)

Key=3

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
														

Binary Search(Another Example)

Key=3

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
														

Binary Search(Another Example)

Key=3

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
														

Binary Search(Another Example)

Key=3

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
														

Binary Search(Another Example)

Key=3

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
														

Binary Search(Another Example)

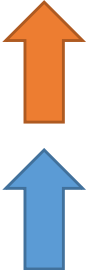
Key=3

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
														

Binary Search(Another Example)

Key=3

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Binary Search(Another Example)

Key=3

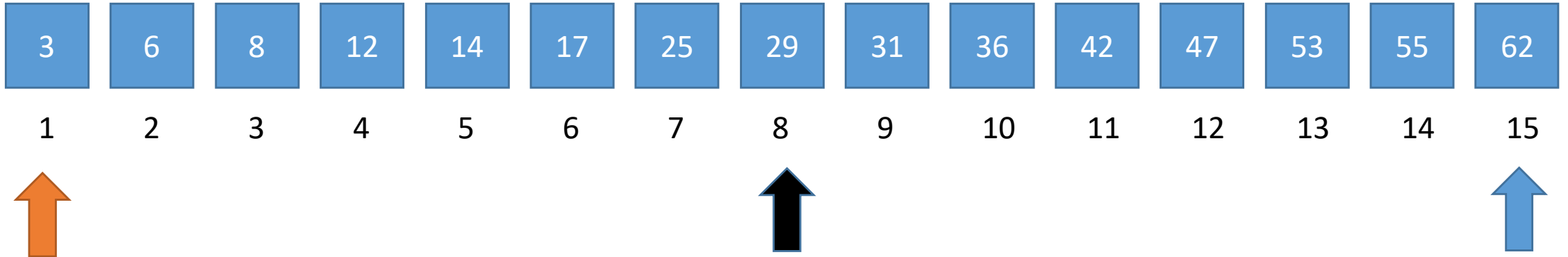
3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



RETURN 1

Binary Search(Last Example)

Key=100



Binary Search(Last Example)

Key=100

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



Binary Search(Last Example)

Key=100

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
								↑			↑			↑

Binary Search(Last Example)

Key=100

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
												↑		↑

Binary Search(Last Example)

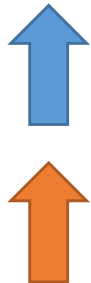
Key=100

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
												↑	↑	↑

Binary Search(Last Example)

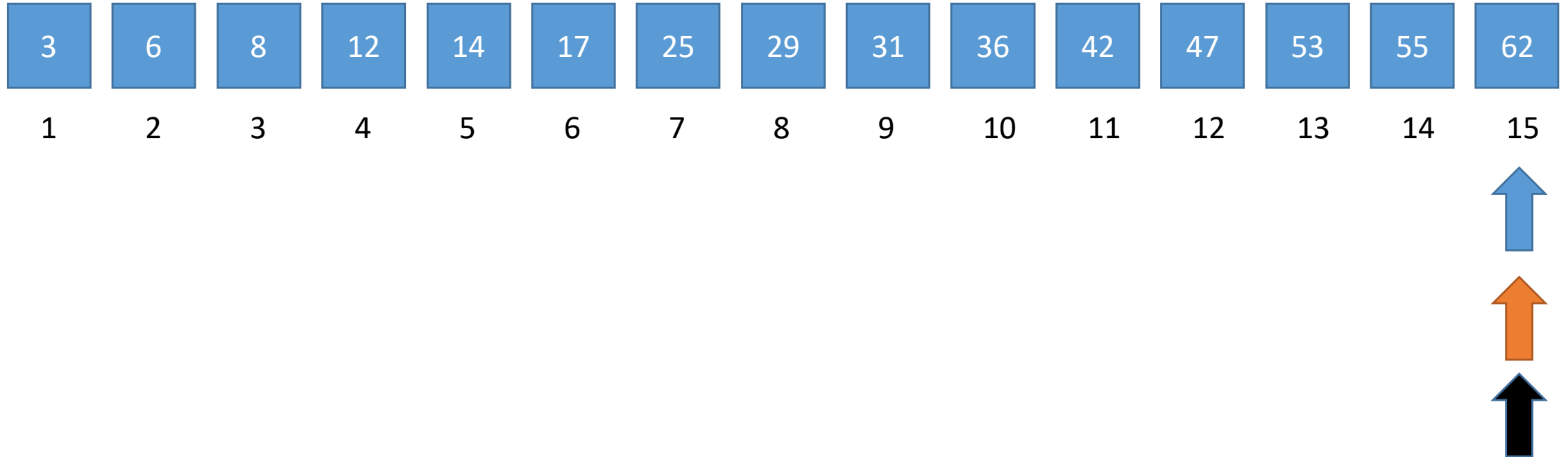
Key=100

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15



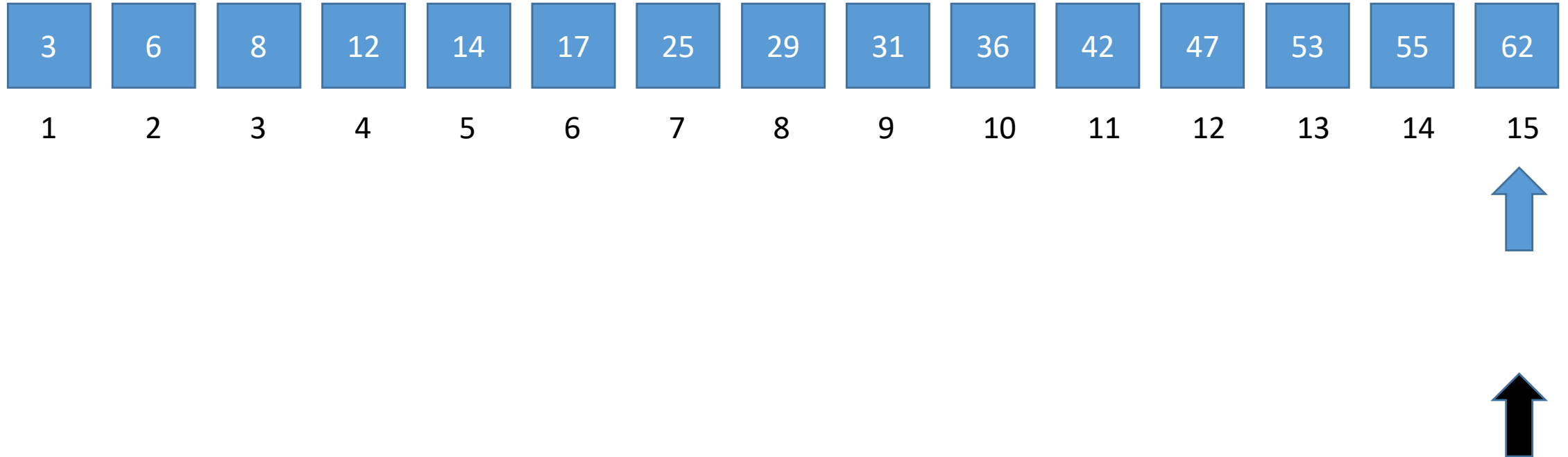
Binary Search(Last Example)

Key=100



Binary Search(Last Example)

Key=100



Binary Search(Last Example)

Key=100

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

RETURN -1



Binary Search (Iterative Pseudocode)

Let us try to write the pseudocode iteratively

Binary Search (Iterative Pseudocode)

Let us try to write the pseudocode iteratively

```
int BinarySearch (int A[], int skey){  
    int low=0, high=SIZE-1, middle;  
    while(low <= high){  
        middle = (low+high)/2;  
        if (skey == A[middle])  
            return middle;  
        else if(skey < A[middle])  
            high = middle - 1;  
        else  
            low = middle + 1;  
    }  
    return -1;  
}
```

Binary Search (Recursive Pseudocode)

Let us try to write the pseudocode recursively

Binary Search (Recursive Pseudocode)

Let us try to write the pseudocode recursively

```
RBmSearch(A, l, h, key):  
1   if (l < h)  
2       return -1  
3   else  
4       mid =  $\frac{l+h}{2}$   
5       if key == A[mid]  
6           return mid  
7       if key < A[mid]  
8           return RBmSearch(A, l, mid-1, key)  
9       if key > A[mid]  
10          return RBmSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9



RBinSearch(A,1,9,4)

RBinSearch(A, l, h, key):

1 if (l < h)

2 return -1

3 else

4  $mid = \frac{l+h}{2}$

5 if key == A[mid]

6 return mid

7 if key < A[mid]

8 return RBinSearch(A, l, mid-1, key)

9 if key > A[mid]

10 return RBinSearch(A, mid+1, h, key)

1 2 3 4 5 6 7 8 9

↑ ↑ ↑

1 2 3 4 5 6 7 8 9

↑ ↑ ↑

RBmSearch(A, l, h, key):

$$1 \quad \text{if } (l \leq h)$$

2 return -1

3 else

4 $\rightarrow$ mid = $\frac{l+h}{2}$

5 if $key == A[mid]$

6 return mid

7 if $key < A[mid]$

8 return RBinSearch(A, l, mid-1, key)

9 if $key > A[mid]$

```
10 return RBinSearch(A, mid+1, h, key)
```


1 2 3 4 5 6 7 8 9

↑ ↑ ↑

1 2 3 4 5 6 7 8 9

↑ ↑ ↑

RBmSearch(A, l, h, key):

$$1 \quad \text{if } (l \leq h)$$

2 return -1

3 else

$$4 \quad \text{mid} = \frac{l+h}{2}$$

5  if key == A[mid]

6 return mid

7 if $key < A[mid]$

8 return RBinSearch(A, l, mid-1, key)

9 if $key > A[mid]$

```
10 return RBinSearch(A, mid+1, h, key)
```


1 2 3 4 5 6 7 8 9

↑ ↑ ↑



```

1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7  → if key < A[mid]
8      return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)

```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9

↑ ↑

RBinSearch(A,1,4,4)

RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7      if key < A[mid]
8          → return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9

↑ ↑

RBinSearch(A,1,4,4)

RBinSearch(A, l, h, key):

```
1 → if (l < h)
2     return -1
3     else
4         mid =  $\frac{l+h}{2}$ 
5         if key == A[mid]
6             return mid
7         if key < A[mid]
8             return RBinSearch(A, l, mid-1, key)
9         if key > A[mid]
10            return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9
↑ ↑ ↑
1 2 3 4 5 6 7 8 9

RBinSearch(A,1,4,4)

RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7      if key < A[mid]
8          return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9

↑ ↑ ↑

RBinSearch(A,1,4,4)

RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      → if key == A[mid]
6          return mid
7      if key < A[mid]
8          return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9
↑ ↑ ↑

RBinSearch(A,1,4,4)

RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7  → if key < A[mid]
8      return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9
↑ ↑ ↑

RBinSearch(A,1,4,4)

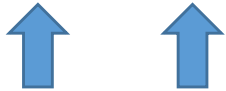
RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7      if key < A[mid]
8          return RBinSearch(A, l, mid-1, key)
9  → if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```


Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9



RBinSearch(A, 3, 4, 4)

RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7      if key < A[mid]
8          return RBinSearch(A, l, mid-1, key)
9  → if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9



RBinSearch(A, 3, 4, 4)

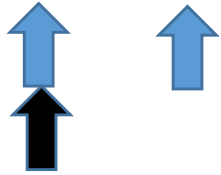
RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7      if key < A[mid]
8          return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         ➡ return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9



RBinSearch(A,3,4,4)

RBinSearch(A, l, h, key):

1 if ($l < h$)

2 return -1

3 else

➔ 4 $mid = \frac{l+h}{2}$

5 if $key == A[mid]$

6 return mid

7 if $key < A[mid]$

8 return RBinSearch(A, l, mid-1, key)

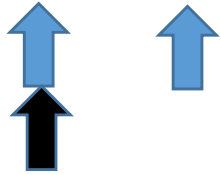
9 if $key > A[mid]$

10 return RBinSearch(A, mid+1, h, key)

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9



RBinSearch(A,3,4,4)

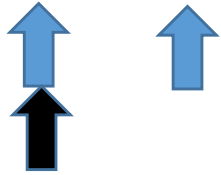
RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7      if key < A[mid]
8          return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```

Binary Search (Recursive Pseudocode)

1	3	4	6	7	10	15	16	20
---	---	---	---	---	----	----	----	----

1 2 3 4 5 6 7 8 9



Returns index 3

RBinSearch(A, l, h, key):

```
1  if (l < h)
2      return -1
3  else
4      mid =  $\frac{l+h}{2}$ 
5      if key == A[mid]
6          return mid
7      if key < A[mid]
8          return RBinSearch(A, l, mid-1, key)
9      if key > A[mid]
10         return RBinSearch(A, mid+1, h, key)
```



Time Complexity Analysis

We can show that:

- Worst Case : $O(\log n)$ – first or last element
- Average Case : $O(\log n)$
- Best Case : $O(1)$ – middle element



THANK YOU